Unit 5

Scrum Methodology, DevOps & Extreme Programming

5.1 Experiment 1: Scrum Method and Framework

5.1.1 Scrum Framework and Its Scope

Scrum is an Agile framework for developing, delivering, and sustaining complex products [15]. It is lightweight, simple to understand, but difficult to master. For the Sanskrit NLP Framework project, Scrum provides the structural backbone for iterative development while maintaining rigorous quality standards for Panini grammar accuracy.

The scope of Scrum in this project includes:

- Managing development of morphological analyzer and semantic parser
- Coordinating between NLP engineers and Sanskrit linguists
- Ensuring regular delivery of working increments
- Providing transparency to stakeholders

5.1.2 Project Management Activities in Scrum

Table 5.1: Scrum Project Management Activities

Activity	**Frequency**	**Purpose**
Sprint Planning	Start of each sprint	Define sprint goal and select backlog items
Daily Scrum	Every day (15 min)	Synchronize activities and identify blockers

Sprint Review	End of each sprint	Inspect increment and adapt backlog
Sprint Retrospective	End of each sprint	Improve processes and team performance
Backlog Refinement	Throughout sprint	Clarify and estimate future backlog items

5.1.3 Sprint Backlog

The Sprint Backlog is a highly visible, real-time picture of the work the Development Team plans to accomplish during the Sprint [15].

Table 5.2: Sprint Backlog Example (Sprint 5)

Task ID	**Description**	**Owner**	**Estimate (hrs)**
SB-01	Implement remaining 200 Panini sutras	NLP Eng 1 + Linguist	40
SB-02	Fix sandhi splitting for 15 edge cases	NLP Eng 2	16
SB-03	Complete visual parse tree backend	NLP Eng 1	8
SB-04	Add authentication to API	NLP Eng 3	8
SB-05	Write integration tests for new endpoints	QA Lead	8
SB-06	Update API documentation	NLP Eng 3	4
Total			**84 hours**

5.1.4 Sprint Review

The Sprint Review is held at the end of the Sprint to inspect the Increment and adapt the Product Backlog [15].

Table 5.3: Sprint Review Structure

Component	**Duration**	**Participants**
Product Owner presents completed work	15 min	All stakeholders

Team demonstrates new functionality	20 min	Development Team
Stakeholders provide feedback	15 min	Stakeholders
Product Owner updates backlog	10 min	Product Owner + Team

5.1.5 Retrospective Perspective

The Sprint Retrospective is an opportunity for the Scrum Team to inspect itself and create a plan for improvements to be enacted during the next Sprint [15].

Table 5.4: Sprint Retrospective Structure

Phase	**Activity**	**Output**
Set the stage	Review sprint goals and outcomes	Shared context
Gather data	What went well? What went wrong?	Data points
Generate insights	Root cause analysis	Improvement hypotheses
Decide what to do	Select top 1-3 improvements	Action items
Close the retrospective	Review action items	Commitment to next sprint

5.1.6 Best Practices of Scrum

Table 5.5: Scrum Best Practices for Sanskrit NLP Project

Best Practice	**Implementation**
Definition of Done	Code reviewed, tests pass, documentation updated, linguist validated
Time-boxed events	Sprint: 2 weeks, Daily Scrum: 15 min, Planning: 2 hours
Cross-functional team	Engineers + Linguist + QA working together
Visible progress	Story board and burndown chart displayed publicly
Sustainable pace	No overtime; 40-hour work weeks
Impediment removal	Scrum Master responsible for blocker resolution
Continuous improvement	Action items from retrospectives tracked and implemented

5.1.7 Roles in Scrum

Table 5.6: Scrum Roles and Responsibilities

Role	Person	Key Responsibilities
Product Owner	Lead Sanskrit Linguist (Dr. Sharma)	Maximize value; manage backlog; prioritize Panini sutras
Scrum Master	Project Manager	Facilitate ceremonies; remove blockers; coach team
Development Team	3 NLP Engineers + 1 QA + 1 Linguist	Deliver potentially shippable increment each sprint

5.1.8 Slack in Scrum

Slack refers to buffer time intentionally built into the sprint to handle unforeseen issues, technical debt, or improvement work.

Table 5.7: Slack Allocation in Sprints

Sprint Purpose	Total Capacity (hrs)	Slack (hrs)
Sprint 1 Team onboarding	80	8
Sprint 2 Learning curve	80	4
Sprint 3 Technical debt	80	4
Sprint 4 Documentation	80	4
Sprint 5 Integration testing	80	8

5.1.9 Ten Minute Build

Ten-minute build is an XP practice that ensures the entire system can be built and all tests can run within 10 minutes [14].

Table 5.8: Ten Minute Build Implementation

Component Target	Current Time
Code compilation < 1 minute	45 seconds
Unit tests (105 tests) < 2 minutes	90 seconds
Integration tests < 2 minutes	120 seconds
Lint and style checks < 30 seconds	30 seconds
Docker image build < 4 minutes	180 seconds
Total **Under 10 min**	**465 seconds (7.75 min)**

5.1.10 Continuous Integration

Continuous Integration (CI) is the practice of merging all developer working copies to a shared mainline several times a day [14].

Table 5.9: CI Pipeline Implementation

Stage	Tool	Trigger
Code commit	Git	Developer push
Linting	flake8, black	On push to any branch
Unit testing	pytest	On push to feature/develop
Integration testing	pytest + API	On PR to develop
Build	Docker	On merge to develop
Deploy (staging)	GitHub Actions	On merge to develop

5.2 Experiment 2: Weekly Cycle, Pair Programming, Coding Standards

5.2.1 Weekly Cycle Structure

Table 5.10: Weekly Agile Cycle

Day	Morning (9:30-12:30)	Afternoon (1:30-5:30)
Monday	Sprint Planning (first week) / Daily Scrum + Coding	Coding + Pair programming sessions
Tuesday	Daily Scrum + Coding	Code review + Documentation
Wednesday	Daily Scrum + Pair programming (complex rules)	Unit testing + Integration testing
Thursday	Daily Scrum + Coding	Code review + Linguist validation
Friday	Daily Scrum + Backlog refinement	Sprint Review (last week) / Retrospective

5.2.2 Pair Programming Schedule

Table 5.11: Pair Programming Schedule

Session	Pair	Focus Area
Monday 2-4 PM	NLP Engineer 1 + NLP Engineer 2	Sandhi rule implementation
Tuesday 2-4 PM	NLP Engineer 2 + Linguist	Panini sutra validation
Wednesday 10-12 PM	NLP Engineer 3 + NLP Engineer 1	API error handling
Thursday 2-4 PM	NLP Engineer 1 + Linguist	Complex morphological cases
Friday 10-12 PM	NLP Engineer 2 + NLP Engineer 3	Performance optimization

5.2.3 Coding Standards (Python – PEP 8 with Extensions)

Table 5.12: Coding Standards

Standard	Description	Tool
Naming conventions	snake_case for variables/functions; PascalCase for classes	flake8
Line length	Maximum 88 characters	black (line length 88)
Docstrings	Google style docstrings for all public functions	pydocstyle

Type hints	Required for all function parameters and returns	mypy (strict mode)
Imports	Grouped: standard library, third-party, local	isort
Testing	Every function must have corresponding unit test	pytest (minimum 80% coverage)

5.2.4 Code Quality Enforcement

```
Pre-commit hooks configuration (.pre-commit-config.yaml):
  - flake8: style guide enforcement (zero warnings)
  - black: automatic code formatting
  - isort: import ordering
  - mypy: static type checking (strict mode)
  - pytest: unit tests (85% coverage minimum)
```

5.3 Experiment 3: DevOps using Docker

5.3.1 Introduction to DevOps

DevOps is a set of practices that combines software development (Dev) and IT operations (Ops) to shorten the development lifecycle while delivering features, fixes, and updates frequently [14]. For the Sanskrit NLP project, DevOps enables:

- Consistent environment across development, staging, and production
- Automated testing and deployment
- Faster feedback loops
- Reduced deployment failures

5.3.2 Docker Implementation Strategy

The Sanskrit NLP Framework is containerized using Docker to ensure consistent execution across environments.

Dockerfile (Production)

```
FROM python:3.10-slim AS base
WORKDIR /app
COPY requirements.txt .
```

```
RUN pip install --no-cache-dir -r requirements.txt
COPY src/ ./src/
COPY models/ ./models/
COPY data/rules/ ./data/rules/
EXPOSE 8000
CMD ["uvicorn", "src.api.main:app", "--host", "0.0.0.0", "--port"
, "8000"]
```

Docker Compose (Development Environment)

```
version: '3.8'
services:
  api:
    build: .
    ports:
      - "8000:8000"
    volumes:
      - ./src:/app/src
      - ./data:/app/data
    environment:
      - ENVIRONMENT=development
      - LOG_LEVEL=debug
    depends_on:
      - redis
  redis:
    image: redis:7-alpine
    ports:
      - "6379:6379"
  tests:
    build: .
    command: pytest tests/ -v --cov=src
    depends_on:
      - api
```

5.3.3 Docker Images and Containers

Table 5.13: Docker Images for Sanskrit NLP Project

Image Name	Purpose	Size

sanskrit-nlp-api:latest	Production API server	1.2 GB
sanskrit-nlp-api:dev	Development with hot-reload	1.4 GB
sanskrit-nlp-tests:latest	Test runner with pytest	1.2 GB
sanskrit-nlp-batch:latest	Batch processing for large corpora	1.3 GB

5.3.4 CI/CD Pipeline with Docker and GitHub Actions

Table 5.14: CI/CD Pipeline Stages

Stage	**Docker Command**	**Trigger**
Build	docker build -t sanskrit-nlp-api:$TAG .	On push to develop
Test	docker run sanskrit-nlp-tests:latest	On PR to main
Push	docker push ghcr.io/sanskrit-nlp/api:$TAG	On merge to main
Deploy (Staging)	docker compose -f docker-compose.staging.yml up -d	On merge to develop
Deploy (Prod)	docker compose -f docker-compose.prod.yml up -d	On tag release/v*

5.3.5 Docker Hub / GitHub Container Registry Strategy

- **Registry**: GitHub Container Registry (GHCR) – integrated with CI/CD
- **Image naming**: `ghcr.io/sanskrit-nlp/sanskrit-nlp-api:latest`
- **Tagging strategy**:
  - `latest` – most recent build from develop branch
  - `v1.0.0` – versioned release tags
  - `sha-xxxxx` – commit hash for traceability
- **Retention**: Keep last 10 images per repository

5.4 Experiment 4: XP Programming – Process Methodology, Framework and Limitations

5.4.1 Introduction to Extreme Programming (XP)

Extreme Programming (XP) is an Agile software development methodology that aims to produce higher quality software and higher quality of life for the development team [14]. XP emphasizes technical excellence and customer satisfaction.

5.4.2 XP Process Methodology

Table 5.15: XP Practices for Sanskrit NLP Project

XP Practice	Implementation	Benefit
Planning Game	User stories with acceptance criteria	Clear requirements
Small Releases	Weekly API updates	Fast feedback
Metaphor	”Panini grammar as rule engine”	Shared understanding
Simple Design	Minimal code, YAGNI principle	Reduced complexity
Testing (TDD)	Write tests before code	High confidence
Refactoring	Monthly code cleanup	Maintainable code
Pair Programming	Two engineers + linguist for complex rules	40% error reduction
Collective Ownership	Any engineer can fix any bug	No silos
Continuous Integration	GitHub Actions on every commit	Early defect detection
Sustainable Pace	40-hour weeks	Consistent velocity
On-site Customer	Linguist full-time	Immediate clarifications
Coding Standards	PEP 8 + extensions	Consistent style

5.4.3 XP Framework

5.4.4 XP Framework Limitations

Table 5.16: XP Limitations and Mitigation Strategies

Limitation	Mitigation for Sanskrit NLP Project
Requires on-site customer	Sanskrit linguist available full-time; budget allocated
Pair programming is resource intensive	Used only for complex Panini rules; not for all tasks
Difficult for large teams	Team size limited to 6 members
Requires cultural shift	Gradual adoption over 3 sprints with coaching
Documentation can suffer	Dedicated documentation sprint every 4 sprints
Not suitable for fixed-price contracts	Hybrid approach: fixed scope for core Panini rules
Scales poorly across multiple teams	Single team for entire project

5.4.5 XP vs Scrum vs Lean Comparison

Table 5.17: Comparison of Agile Methodologies Used

Aspect	Scrum	XP	Lean
Primary focus	Project management	Engineering practices	Waste elimination
Roles	Product Owner, Scrum Master, Team	No explicit roles	No explicit roles
Ceremonies	Sprint Planning, Review, Retro, Daily Scrum	Planning Game, Stand-up	Continuous improvement
Key practices	Backlog, Burndown, Timeboxing	TDD, Pair programming, CI	Value stream mapping
Artifacts	Product Backlog, Sprint Backlog, Increment	User stories, Tests, Code	Kanban board

Best for	Complex product development	Technical excellence
Process optimization		

5.4.6 Hybrid Approach for Sanskrit NLP Project

The project uses a hybrid approach combining:

- **Scrum** for project management (sprints, roles, ceremonies)
- **XP** for engineering practices (TDD, pair programming, CI)
- **Lean** for waste elimination (automated documentation, value stream mapping)

5.4.7 Process Methodology Summary

Table 5.18: Process Methodology Summary

Phase	**Methodology**	**Key Activities**
Requirements	Scrum	Product backlog, user stories, prioritization
Development	XP	TDD, pair programming, refactoring
Integration	XP + DevOps	CI/CD, ten-minute build
Review	Scrum	Sprint review, stakeholder feedback
Improvement	Lean	Retrospective, waste elimination

5.4.8 Continuous Integration Pipeline (Complete)

```
Push to any branch
       |
       v
+--------------+
|    Lint      | (flake8, black --check, isort)
+--------------+
       |
       v
+--------------+
```

```
+------------------+
|   User Stories   |
+------------------+
         |
         v
+------------------+
|   Release Plan   |
+------------------+
         |
         v
+------------------+     +------------------+
|    Iteration     |---->|   Acceptance     |
|    Planning      |     |    Tests         |
+------------------+     +------------------+
         |
         v
+------------------+
|   Pair Coding    |
+------------------+
         |
         v
+------------------+     +------------------+
|    Unit Tests    |---->|   Integration    |
|    (TDD)         |     |    (CI)          |
+------------------+     +------------------+
         |
         v
+------------------+
|   Small Release  |
+------------------+
```

Figure 5.1: XP Process Flow for Sanskrit NLP Project

```
|  Type Check  | (mypy --strict)
+--------------+
       |
       v
+--------------+
|  Unit Tests  | (pytest --cov=src --cov-fail-under=85)
+--------------+
       |
       v
+--------------+
| Integration  | (pytest tests/integration)
+--------------+
```

```
        |
        v
+--------------+
| Build Docker | (docker build -t sanskrit-nlp-api .)
+--------------+
        |
        v
+--------------+
| Deploy Staging| (if branch = develop or main)
+--------------+
```

5.4.9 Weekly Cycle Effectiveness Metrics

Table 5.19: Weekly Cycle Effectiveness Over Time

Metric	**Week 1**	**Week 4**	**Week 8**
Story points completed	18	24	26
Defects discovered	12	7	4
Code review turnaround	8 hours	4 hours	2 hours
Pair programming sessions	2	4	6
Linguist feedback time	24 hours	12 hours	4 hours

5.4.10 Pair Programming Benefits Quantified

Table 5.20: Pair Programming Benefits

Benefit	**Before Pair Programming**	**After Pair Programming**
Rule implementation errors	15%	6%
Code review defects	8 per sprint	3 per sprint
Knowledge transfer time	2 weeks	3 days
Bug fix time (critical)	8 hours	3 hours

5.4.11 Conclusion for Unit 5

The Scrum framework successfully structured the Sanskrit NLP project with defined roles (Product Owner, Scrum Master, Development Team), artifacts (Product Backlog, Sprint Back-

log, Increment), and ceremonies (Sprint Planning, Daily Scrum, Sprint Review, Retrospective) [15]. Best practices including Definition of Done, time-boxed events, and sustainable pace were consistently followed. The ten-minute build was achieved at 7.75 minutes, meeting the XP standard. Continuous Integration via GitHub Actions provided rapid feedback on code quality. Docker containerization ensured consistent deployment across environments, and the CI/CD pipeline automated testing and deployment [14]. XP practices, particularly pair programming and TDD, reduced implementation errors by 60%. The hybrid Agile approach successfully balanced the need for rigorous Panini grammar accuracy with flexible, iterative development, delivering working software every two weeks while maintaining linguistic precision.

Table 5.21: Final Project Summary

Parameter	**Value**
Total Project Duration (Baseline)	218 days
Total Project Duration (Crashed)	200 days
Project Budget	INR 33–41 Lakhs
NPV (10% discount)	INR +37.72 Lakhs
Payback Period	2.14 years
Critical Path	A $\rightarrow$ C $\rightarrow$ D $\rightarrow$ E $\rightarrow$ G $\rightarrow$ H $\rightarrow$ I
Team Size	10 members
Sprint Duration	2 weeks
Average Velocity	26 story points per sprint
Code Coverage	85%
Unit Test Pass Rate	87.6% $\rightarrow$ 95% (target)